

Welcome to LangChain

Contents

- [Getting Started](#)
- [Modules](#)
- [Use Cases](#)
- [Reference Docs](#)
- [LangChain Ecosystem](#)
- [Additional Resources](#)

Large language models (LLMs) are emerging as a transformative technology, enabling developers to build applications that they previously could not. But using these LLMs in isolation is often not enough to create a truly powerful app - the real power comes when you are able to combine them with other sources of computation or knowledge.

This library is aimed at assisting in the development of those types of applications. Common examples of these types of applications include:

Question Answering over specific documents

- [Documentation](#)
- End-to-end Example: [Question Answering over Notion Database](#)

Chatbots

- [Documentation](#)
- End-to-end Example: [Chat-LangChain](#)

Agents

- [Documentation](#)
- End-to-end Example: [GPT+WolframAlpha](#)

Getting Started

Checkout the below guide for a walkthrough of how to get started using LangChain to create an Language Model application.

- [Getting Started Documentation](#)

Modules

There are several main modules that LangChain provides support for. For each module we provide some examples to get started, how-to guides, reference docs, and conceptual guides. These modules are, in increasing order of complexity:

- **Prompts:** This includes prompt management, prompt optimization, and prompt serialization.
- **LLMs:** This includes a generic interface for all LLMs, and common utilities for working with LLMs.
- **Document Loaders:** This includes a standard interface for loading documents, as well as specific integrations to all types of text data sources.
- **Utils:** Language models are often more powerful when interacting with other sources of knowledge or computation. This can include Python REPLs, embeddings, search engines, and more. LangChain provides a large collection of common utils to use in your application.
- **Chains:** Chains go beyond just a single LLM call, and are sequences of calls (whether to an LLM or a different utility). LangChain provides a standard interface for chains, lots of integrations with other tools, and end-to-end chains for common applications.
- **Indexes:** Language models are often more powerful when combined with your own text data - this module covers best practices for doing exactly that.
- **Agents:** Agents involve an LLM making decisions about which Actions to take, taking that Action, seeing an Observation, and repeating that until done. LangChain provides a standard interface for agents, a selection of agents to choose from, and examples of end to end agents.
- **Memory:** Memory is the concept of persisting state between calls of a chain/agent. LangChain provides a standard interface for memory, a collection of memory implementations, and examples of chains/agents that use memory.
- **Chat:** Chat models are a variation on Language Models that expose  [latest](#)  rather than working with raw text. they work with messages. LangChain provides a

[Skip to main content](#)

Use Cases

The above modules can be used in a variety of ways. LangChain also provides guidance and assistance in this. Below are some of the common use cases LangChain supports.

- **Agents:** Agents are systems that use a language model to interact with other tools. These can be used to do more grounded question/answering, interact with APIs, or even take actions.
- **Chatbots:** Since language models are good at producing text, that makes them ideal for creating chatbots.
- **Data Augmented Generation:** Data Augmented Generation involves specific types of chains that first interact with an external datasource to fetch data to use in the generation step. Examples of this include summarization of long pieces of text and question/answering over specific data sources.
- **Question Answering:** Answering questions over specific documents, only utilizing the information in those documents to construct an answer. A type of Data Augmented Generation.
- **Summarization:** Summarizing longer documents into shorter, more condensed chunks of information. A type of Data Augmented Generation.
- **Evaluation:** Generative models are notoriously hard to evaluate with traditional metrics. One new way of evaluating them is using language models themselves to do the evaluation. LangChain provides some prompts/chains for assisting in this.
- **Generate similar examples:** Generating similar examples to a given input. This is a common use case for many applications, and LangChain provides some prompts/chains for assisting in this.
- **Compare models:** Experimenting with different prompts, models, and chains is a big part of developing the best possible application. The ModelLaboratory makes it easy to do so.

Reference Docs

All of LangChain's reference documentation, in one place. Full documentation on all methods, classes, installation methods, and integration setups for LangChain.

- [Reference Documentation](#)



[Skip to main content](#)

LangChain Ecosystem

Guides for how other companies/products can be used with LangChain

- [LangChain Ecosystem](#)

Additional Resources

Additional collection of resources we think may be useful as you develop your application!

- [LangChainHub](#): The LangChainHub is a place to share and explore other prompts, chains, and agents.
- [Glossary](#): A glossary of all related terms, papers, methods, etc. Whether implemented in LangChain or not!
- [Gallery](#): A collection of our favorite projects that use LangChain. Useful for finding inspiration or seeing how things were done in other applications.
- [Deployments](#): A collection of instructions, code snippets, and template repositories for deploying LangChain apps.
- [Discord](#): Join us on our Discord to discuss all things LangChain!
- [Tracing](#): A guide on using tracing in LangChain to visualize the execution of chains and agents.
- [Production Support](#): As you move your LangChains into production, we'd love to offer more comprehensive support. Please fill out this form and we'll set up a dedicated support Slack channel.